

How the notes pipeline works — one agent at a time

This document explains every "agent" in the Agentic Notes pipeline: what it does, exactly how it does it, what it's good at, and where it can still fail. It's written for a human reading it later (including future-you), not for the model.

The whole pipeline lives in `backend/agent.py` and is driven by one function, `run_agent(...)`, which runs the agents in order and streams progress events to the frontend as it goes. Think of it as an assembly line: source text goes in one end, and notes + quiz + flashcards come out the other, with several checkpoints in between where the work gets checked and improved.

The big picture first

```
Source text
|
v
0. Gatekeeper --reject--> "not academic material" (stop here)
| pass
v
  Build a search index over the source (chunk + embed + BM25)
  |
  v
1. Planner --> outline + checklist + difficulty + format
  |
  v
2. Writer --> first draft of the notes (streamed live to the screen)
  |
  v
3. Critique --> score /10 + list of unsupported claims + missing topics
  |
  +-- good enough? --> skip ahead
  |
  +-- needs work? --> 4. Revise --> back to Critique (loop, max 2 times)
  |
  v
5. Title agent --> short name for this session
  |
  v
6. Quiz agent --> 5 MCQs --> 7. Quiz verifier double-checks the answer key
  |
  v
8. Flashcard agent --> 8 flashcards
  |
  v
Done -- everything streamed to the browser as it's produced
```

Two important ideas run underneath all of this:

- 1 **Retrieval-Augmented Generation (RAG).** The model never sees the raw, full document. Instead, the source is chopped into small overlapping passages ("chunks"), and every agent that needs source material asks a **retriever** for the most relevant chunks first. This is what keeps notes grounded in the real document instead of the model just making things up from memory.
 - 2 **Model routing.** Not every agent uses the same "brain." The steps where quality really matters (planning, writing, critiquing) use the strong model you picked in the UI. The mechanical, low-stakes steps (checking if the topic is academic, naming the session, writing quiz questions) use a cheaper, faster helper model. This is explained fully in its own section near the end — it's one of the more important design decisions in the whole system.
-

Agent 0 — The Gatekeeper

File: `classify_academic()` in `agent.py`

Job: Look at the source text and decide: is this actually study material? The product is called a *study* notes generator, so it refuses to build notes for celebrity gossip, sports scores, ads, or random chit-chat.

How it works: It sends the first 4,000 characters of the source to the model with a strict instruction — accept science, maths, computer science, engineering, medicine, humanities, history, economics, law, languages; reject entertainment, marketing, casual content. The model replies with a small JSON verdict: `{"academic": true/false, "subject": "...", "reason": "..."}.` If the answer is "no," the whole pipeline stops immediately and the UI shows the reason — no notes are generated, no tokens are wasted downstream.

Strengths:

- Cheap and fast — only reads 4k characters, uses the small helper model.
- Fails safe: if the model's answer can't be parsed as JSON, the gatekeeper *lets the content through* rather than blocking it. A confused gatekeeper is treated as "not sure, so don't get in the way."
- Runs before the expensive retrieval index is even built, so a rejected topic costs almost nothing.

Limitations:

- It only reads the *first* 4,000 characters. A 100-page document that opens with a boring cover page or table of contents, but is genuinely academic later, could theoretically be misjudged — though in practice this hasn't been an issue since real documents establish their subject quickly.
 - It's a single model call with no second opinion — an unusual or borderline topic (e.g., a philosophical essay that reads casually) could be judged either way. There's no appeals process, though the safe-parse fallback keeps the blast radius small.
-

The Retriever (not an "agent," but everything depends on it)

Files: `retriever.py` (public interface) and `retrieval/` folder (`bm25.py`, `semantic.py`, `fusion.py`, `hybrid.py`, `config.py`)

Technically the retriever doesn't call the language model at all — it's a search engine built fresh over each document. But every other agent's quality depends entirely on what it hands them, so it deserves its own explanation.

How it works, step by step:

- 1 **Chunking.** The document is split into passages of roughly 700 characters, with 120 characters of overlap between neighbors (so a sentence that happens to fall on a chunk boundary isn't cut off with no context). Each chunk gets a stable ID number.
- 2 **Two independent search methods run over those chunks:**
- 3 **BM25** — a classic keyword-matching algorithm (the same family of algorithm search engines used for decades). It's good at exact-term matches: if the notes need to talk about "mitochondria," BM25 finds every chunk that literally contains that word.
- 4 **Semantic search** — the chunks are converted into embeddings (numerical "meaning fingerprints") using Google's Gemini embedding model, and search compares meaning, not just exact words. This finds passages that are relevant even if they use different wording than the query.
- 5 **Fusion (Reciprocal Rank Fusion, RRF).** The two ranked lists get merged into one combined ranking, so a chunk that both methods agree on ranks higher than one only one method liked. This hybrid approach is deliberately more robust than either method alone — semantic search alone can miss exact technical terms, and keyword search alone can miss a paraphrase.
- 6 **The result:** a `retrieve(query, k)` call returns the top-`k` chunks in relevance order, each with a small integer ID like `[3]` that the writer can cite.

There's also a `sample()` method, which is different from `retrieve()` — instead of relevance-ranking, it takes an *even spread* of chunks across the whole document (like reading every 10th page). This is used specifically so the Planner and Critique agents get a breadth view of the entire document, not just the parts that best match one search query.

Strengths:

- Hybrid retrieval covers both "the exact term" and "the same idea, different words" — genuinely more reliable than picking one method.
- If Gemini embeddings are unavailable (e.g., no API key, or a transient failure), it logs a clear warning and automatically falls back to BM25-only rather than crashing.
- Retrieval is fully re-buildable per document — there's no shared state or contamination between different users' uploads.

Limitations:

- Chunking is purely by character count — it doesn't know about headings, paragraphs, or sentence structure. A chunk could start mid-sentence. (A structure-aware chunker that respects headings/paragraphs is a known improvement that hasn't been built yet.)
- Semantic search depends on an external API call to Gemini for embeddings, which adds latency and a point of failure (mitigated by the BM25 fallback, but fallback is still a downgrade).

- There is currently no reranking step after fusion — the `_process_candidates()` method exists specifically as a hook for a future cross-encoder reranker, but today it's a pass-through (identity function). This means final ranking quality is capped at what BM25 + embeddings + RRF can achieve alone.
-

Agent 1 — The Planner

File: `plan_outline()`

Job: Read a broad sample of the document and produce a study plan before any notes are written — like an outline a student would sketch before writing an essay.

How it works: It's given an even sample of the document (`retriever.sample(24000)` — up to 24,000 characters spread across the whole thing, not just the start) and asked to return JSON with four fields:

- `outline` — the section headings the notes should have
- `checklist` — specific points that **MUST** be covered, used later to check nothing important got dropped
- `difficulty` — beginner / intermediate / advanced
- `suggested_format` — bullet points, a numbered list, or paragraphs

Critically, **the outline size scales with document size**. A short document gets 3–5 sections. A "substantial" document (over 20,000 characters) is told to produce 5–8 sections. A very large document (over 120,000 characters — think a full book chapter) is told to produce 8–12 sections and explicitly instructed: *"cover ALL of its major topics — do not skip parts of the document."* This single change is what fixed the earlier problem where 100-page uploads lost most of their content — the plan itself now demands full coverage before a single word of notes is written.

The digest scan — how the planner sees 100% of very large documents. For a 100-page document, the 24,000-character sample covers only about 8% of the text — one passage every ~3 pages. That's fine for spotting big themes, but a topic confined to just 2–3 pages could be completely invisible in the sample, and a topic the planner never names is a topic the writer never retrieves. So for documents over 60,000 characters (`DIGEST_DOC_THRESHOLD`), a preliminary step runs first: the **cheap helper model reads the ENTIRE document**, in segments of 25,000 characters, and lists each segment's topics (`digest_document()`). The merged result — a full-coverage "topic inventory" — is handed to the planner with a hard instruction: *your outline MUST cover all major topics in this inventory*. The planner still gets the even sample too (for tone, difficulty, and structure), but coverage decisions now come from a scan that saw every page. Because the scan runs on the helper model, its token cost comes out of a separate free-tier quota, not the main model's budget.

Strengths:

- For big documents, coverage is now decided by a 100% scan of the text, not an 8% sample — a topic that appears on only two pages still makes it into the inventory, and therefore into the outline.
- Seeing an even sample (not just the beginning) means the plan reflects the whole document's shape, not just its introduction.

- The checklist becomes a real accountability tool later — the Critique agent explicitly checks whether every checklist item made it into the final notes.
- Fails soft everywhere: a failed digest segment is skipped (partial inventory still beats none), and there's a sensible hardcoded outline fallback (`Overview / Key Concepts / Important Details / Summary`) if the model's JSON can't be parsed — a single bad response never crashes the whole pipeline.

Limitations:

- The digest adds one helper-model call per 25,000 characters (up to 12 for a maximum-size upload), which adds some seconds of latency and spends helper-quota tokens before writing even starts — the price of genuine full coverage.
- The inventory's quality depends on the helper model's summarization: it reads everything, but if it describes a topic too vaguely, the planner may group it away rather than giving it a proper section. The writer's outline is capped at 8 written sections, so an inventory with 30 distinct topics still has to be condensed.
- The plan is made once and never revisited. If the Critique step later finds a big gap, the fix happens through targeted revision, not by re-running the planner with new information.

Agent 2 — The Writer (and its "long document" alter ego)

Files: `write_notes_stream()` for normal documents, `write_section_stream()` for long ones

Job: Turn retrieved passages into actual, readable study notes — the part the user is staring at as it streams onto the page.

Two different modes, chosen automatically based on document size:

A. Fast path — small/medium documents (under ~20,000 characters). One retrieval call fetches the top 8 most relevant chunks for the whole document (using the outline + checklist + study mode as the search query). The writer gets all of them at once and writes the complete notes in a single streamed pass, following the outline, respecting the target word count, and citing sources like [2][5] as it goes.

B. Map-reduce path — long documents (over ~20,000 characters). This is the fix for the "100-page PDF loses most of its content" problem. Instead of one retrieval for the entire document, **every outline section gets its own dedicated retrieval and its own writing pass.** Concretely:

- For each section in the outline (e.g., "Cell Respiration," "The Krebs Cycle," "ATP Synthesis"), the retriever is asked specifically for chunks relevant to *that* section.
- The writer then writes just that one section, grounded only in its own retrieved passages, streamed to the screen with a live status like *"Writing section 3/8: The Krebs Cycle..."*
- The number of sections is capped at 8 (`SECTION_MAX_COUNT`) so an enormous document can't trigger an unbounded number of model calls and blow through rate limits.

Either way, both writer variants are told the same core rules: follow the outline, hit the target length, use the requested format (bullets/numbered/paragraphs), match the mode (exam/revision/deep study/summary) and

tone (academic/formal/casual/simple), and **cite every claim** to a passage number so it can be traced back to the source.

Strengths:

- The map-reduce approach means the LAST section of a huge document gets just as much dedicated retrieval and writing attention as the first — nothing structurally gets "run out of budget."
- Citations are baked into the prompt as a hard rule, which gives the reader (and the Critique agent) a way to trace any claim back to its source passage.
- Streaming means the user sees progress immediately instead of staring at a blank screen for 30+ seconds.

Limitations:

- **Citations are never actually verified.** The writer is *told* to only cite passage numbers that exist and that support the claim, but nothing currently checks that a [3] next to a sentence really is supported by passage 3. This is the single biggest trust gap in the system right now — a citation could technically be attached to the wrong passage and nothing would catch it before the critique step (and the critique step checks the *claim*, not the *citation number* specifically).
- In the map-reduce path, each section is written in isolation from the others. This is what makes coverage of the whole document possible, but it also means the writer can't easily avoid repeating an idea across two related sections, since it doesn't see what it just wrote in the previous one.
- Section count is capped at 8. An exceptionally content-dense document with a 12-section plan will only get its first 8 sections written — the plan can outline more topics than the writer will actually execute.

Agent 3 — Critique

File: `critique_notes()`

Job: Be the "second pair of eyes" that checks the draft against the real source, before the user ever sees a claim that isn't actually true.

How it works: It compares the notes against a *breadth sample* of the source (`retriever.sample(12000)` — again, spread across the whole document, not just the start) and scores three separate things:

- 1 **Faithfulness** — is every claim actually backed by the source? Anything invented, exaggerated, or distorted gets listed as an `unsupported_claim`.
- 2 **Coverage** — does anything from the checklist or the source seem to be missing? Gaps get listed as `missing_topics`.
- 3 **Quality** — is it clear, well organized, and actually useful for studying?

It returns a score from 1–10, plus the specific list of unsupported claims and missing topics — not just a number, but a **reason** the revision agent can act on directly.

The scoring is deliberately strict about faithfulness. A score of 8+ is only treated as "good enough" if there are zero unsupported claims — even a high score gets forced into revision if the model found even one fabricated statement. This reflects the philosophy of the whole project: a wrong-but-confident note is worse than a note that's missing something.

Strengths:

- The fallback behavior is deliberately paranoid: if the critique's JSON can't be parsed for any reason, the code doesn't assume the notes are fine — it defaults to `needs_revision: true`. Silence is treated as "not verified," never as "verified."
- Comparing against a breadth sample of the source (rather than just the beginning) means faithfulness checking works properly even on the parts of a long document that come after page 50.
- Separating "faithfulness" from "coverage" from "quality" gives the revision agent a precise target instead of a vague "make it better."

Limitations:

- For very long, map-reduce-written documents, the critique compares the *entire* set of notes against a single 12,000-character sample. That sample can't possibly contain everything the notes now cover — a long document with 8 sections and thousands of words of notes is being checked against a fixed-size window into the source. A more precise (but not-yet-built) version of this would check each section against the specific passages *that section* was written from, which is the natural next improvement.
- It's still one model's judgment. There's no independent second critique or human-in-the-loop check — if the critique model itself misses a subtle fabrication, nothing downstream catches it either.

Agent 4 — Revise (the self-correction loop)

File: `revise_notes_stream()`

Job: Take the critique's specific complaints and actually fix them, without throwing away what was already correct.

How it works: The revision prompt is built directly from the critique's structured output — it explicitly lists the unsupported claims to remove/correct, the missing topics to add (with a reminder that they must actually exist in the context), and any other quality issues to fix. It's also given the same retrieved context the writer had, so anything it adds is still grounded in real passages, not invented from memory.

The loop: After a revision, the critique agent runs again on the new version. If it's still not good enough, this repeats — but it's capped at `MAX_REVISION_ROUNDS = 2` so a stubborn problem doesn't loop forever (and burn API tokens forever). The pipeline also keeps track of the **best-scoring version seen across all rounds** — if a revision accidentally makes things worse (which can happen), the pipeline quietly falls back to the earlier, better-scoring version rather than shipping a regression.

Strengths:

- Grounded in the actual retrieved context again, not just "the notes so far" — this stops a revision from fixing one fabrication by introducing another.
- The "keep the best version" safety net means a single bad revision round can't make the final output worse than the version before it.
- Bounded at 2 rounds — protects against runaway cost and latency on a document that just won't satisfy the critique.

Limitations:

- If the source genuinely doesn't contain something the checklist demands, the revise loop has no way to signal "this can't be fixed" — it will keep trying and failing to add content that isn't there, for up to 2 rounds, before the loop simply gives up and ships the best attempt.
 - Two rounds is a real ceiling. A document with several distinct faithfulness problems might need more passes than the budget allows; the tradeoff is intentional (cost and latency vs. thoroughness) but it is a real limitation, not a solved problem.
-

Agent 5 — Title

File: `generate_title()`

Job: Name the session automatically (e.g., "Cellular Respiration Basics") so the user doesn't have to.

How it works: Reads just the first 1,500 characters of the finished notes and asks for a 3–6 word Title Case name. Runs on the cheap helper model — this task genuinely doesn't need a strong model.

Strengths: Fast, cheap, and wrapped in a try/except in the orchestrator — if it fails for any reason, the pipeline just skips having a title rather than failing the whole run.

Limitations: Only reads the beginning of the notes, so on rare occasions the title might reflect the first section rather than the document's true overall theme (usually not an issue since the first section is normally representative).

Agents 6 & 7 — Quiz and Quiz Verifier

Files: `generate_quiz()` and `verify_quiz()`

Job: Turn the finished notes into 5 multiple-choice questions — and then make sure the answer key is actually correct.

Why two separate steps: Generating a quiz and grading it correctly are different skills, and combining them in one pass is where mistakes creep in (a model can write a good question but mark the wrong letter as the answer). So this is explicitly a two-stage process:

- 1 **Generate:** produce exactly 5 questions in a strict plain-text format (Q1) ... A) B) C) D) ... Answer: ... Explanation: ...).
- 2 **Verify:** a second, separate pass re-reads each question against the notes and checks whether the marked answer is actually correct — fixing the Answer letter and Explanation if not, while preserving the exact format.

A safety guard on the verifier: if the verifier's output comes back suspiciously short (less than half the length of the original, or missing the word "Answer" entirely), the code assumes the verification pass degenerated and just keeps the original quiz rather than replacing good output with garbage.

Strengths:

- The verify step genuinely catches a class of error that's easy for a single model pass to make — marking the "obviously right-sounding" option instead of the one actually supported by the notes.
- Runs entirely on the cheap helper model, since this is a mechanical, well-specified task.
- The degenerate-output guard means a mid-pipeline hiccup can't corrupt an otherwise-fine quiz.

Limitations:

- Verification checks the quiz against the *notes*, not the *original source*. If the notes themselves already contain a subtle error, the quiz verifier can only be as correct as the notes — it can't independently catch mistakes that trace back to the writer.
- There's no distractor-quality check — a technically-correct quiz could still have implausible wrong answers (e.g., three options that are obviously wrong at a glance), which would make the question too easy. Nothing currently scores question difficulty or distractor quality.

Agent 8 — Flashcards

File: `generate_flashcards()`

Job: Produce 8 front/back flashcards from the finished notes, in a strict, parseable format (CARD 1 / Front: ... / Back: ...), ready to be reviewed with the app's spaced-repetition scheduler.

Strengths: Simple, fast, runs on the helper model, and the strict text format makes it easy to parse reliably downstream (used for the Again/Good/Easy spaced-repetition scheduling and CSV export elsewhere in the app).

Limitations: Like the quiz, flashcards are generated but never independently verified against the notes the way the quiz answer key is — there's no "flashcard verifier" step. A subtly wrong flashcard back would currently ship without a second check.

The other helper agents (used outside the main pipeline)

A few more single-purpose agents exist in `agent.py` that don't run as part of `run_agent()`, but are called directly by other parts of the app:

- `rewrite_notes()` — condenses notes to roughly half length, expands them to ~1.5x, or just improves clarity, depending on what button the user clicks. Used by the "shorter / longer" controls in the UI.
- `edit_selection()` — applies a specific instruction (like "make this simpler") to just the text a user has highlighted, leaving the rest of the notes untouched. This is the "inline assistant" feature — select text, ask for a change, get back the complete notes with just that passage updated.
- `chat_about_notes_stream()` — a tutor-style chat that answers questions grounded primarily in the generated notes, with the last 6 turns of conversation history included for context. If the notes don't cover something, it's explicitly allowed to fall back to general knowledge, but told to say so.

These all share the same philosophy as the main pipeline — stay grounded in the notes/source wherever possible — but they're simpler, single-shot calls rather than multi-agent loops, because their tasks don't need planning or self-correction.

Model routing: why not every agent uses the same "brain"

File: `helper_model()` in `models.py`, applied throughout `run_agent()`

This is a cross-cutting design decision worth explaining on its own, because it isn't obvious from reading any single agent.

The app runs on **free-tier API keys**, and free tiers have **daily token limits** (not just per-minute ones). A single large document run — especially with the map-reduce writer — can use tens of thousands of tokens. If every one of the 8+ model calls in a run used the same strong model, that model's daily quota would run out quickly, and the *whole app* would stop working until the next day.

The fix: split work across **two separate quotas**.

- **The strong model the user picked** (e.g., Llama 3.3 70B) handles the three steps where output quality is actually decided: **Plan, Write, Critique**.
- **A cheap helper model** (Llama 3.1 8B, or Gemini Flash-Lite if the user picked a Gemini model) — which has its **own, completely separate** daily token budget — handles the mechanical steps: **Gatekeeper, Title, Quiz, Flashcards**.

Because Groq's daily limits are tracked *per model*, this isn't just a cost optimization — it roughly doubles how many documents the app can process per day before hitting a wall, since half the pipeline's calls now draw from a quota that was previously sitting completely unused.

Strengths: Free, effective, and doesn't compromise quality where it matters — the tasks moved to the helper model (checking if something's an academic topic, naming a session, formatting quiz questions) genuinely don't need a large, expensive model to do well.

Limitations: If the user's chosen strong model's daily quota is exhausted, the app has no way to automatically switch them to a fresh model — they have to manually pick a different one from the dropdown

(e.g., switch from the 70B model to the 8B model, which has never been touched that day). There's also a system-wide failover chain (Groq -> Gemini -> Ollama) for outright errors, but it doesn't currently protect against *both* configured providers being out of daily quota on the same day — which is exactly what happened when this was diagnosed: Groq's 70B daily limit and Gemini's daily limit were both exhausted at once.

Honest summary: where this system is strong, and where it still has real gaps

What it's genuinely good at:

- Never silently failing — every agent has a sensible fallback (permissive gatekeeper, safe default plan, "assume revision needed" critique) so a single bad model response can't crash or corrupt the whole run.
- Actually checking its own work — the critique/revise loop is a real self-correction mechanism, not just a single-shot generation with a friendly bow on top.
- Covering entire long documents — the map-reduce rewrite specifically fixed the earlier failure where large uploads silently lost most of their content.
- Being honest about faithfulness — the scoring is deliberately strict about penalizing fabricated claims over almost everything else.

What's still genuinely unverified or missing today:

- **Citations are asserted, never checked.** A [3] next to a claim is not currently confirmed to actually match what passage 3 says.
- **No cross-encoder reranking** after the hybrid search — the retrieval quality is capped at what BM25 + embeddings + RRF can do; the code has a clean seam (`_process_candidates()`) for this to be added later, but it isn't built yet.
- **Critique on long documents checks against a sample, not the specific passages each section was actually written from** — coarser than the writing process it's checking.
- **No quiz distractor-quality or flashcard-answer verification** — only the quiz's *answer key* is double-checked, not the plausibility of wrong options or the correctness of flashcard backs.
- **Daily free-tier quotas are a real ceiling**, not just a hypothetical — this has already caused a real outage in production, and the two-model routing fix reduces but does not eliminate the risk.

This isn't a finished, perfect system — it's a genuinely well-engineered one with clear, honest edges. Knowing exactly where those edges are is what makes it possible to keep improving it deliberately instead of guessing.